

Este eje temático es fundamental para construir la interacción real con el usuario en aplicaciones MVC, manejando datos de forma estructurada y validándolos. Aquí tienes el material de estudio detallado para los 4 encuentros planificados.

Material de Estudio: Modelos, Vistas Avanzadas y Formularios en MVC

Profesor: Jorge Izaguirre

Materia: Programacion .NET

Curso: 6to Año - TÉCNICO EN PROGRAMACIÓN

Parte 1: Conectando Datos con Vistas - Modelos, ViewModels y Binding

(1.1) Repaso y la Necesidad de Estructura

En la introducción a MVC, vimos cómo pasar datos simples del Controlador a la Vista usando *ViewBag* o *ViewData*. Rápidamente mencionamos sus limitaciones:

- **Falta de Seguridad de Tipos:** Son dinámicos (*ViewBag*) o basados en strings (*ViewData*), por lo que el compilador no puede verificar si usamos los nombres correctos o los tipos de datos adecuados. Los errores solo aparecen en tiempo de ejecución.
- **Sin IntelliSense:** El editor no nos ayuda a autocompletar nombres ni a ver las propiedades disponibles.
- **Código Propenso a Errores:** Es fácil cometer errores de tipeo en los nombres de las propiedades/claves.
- **Requiere Casting:** A menudo necesitamos convertir los datos de vuelta al tipo original en la Vista.

Para construir aplicaciones robustas y mantenibles, necesitamos una forma **estructurada y fuertemente tipada** de pasar datos entre el Controlador y la Vista. Aquí es donde entran los Modelos y, más específicamente, los ViewModels.

(1.2) El Modelo (Model) y las Clases POCO

Recordemos que el **Modelo** en MVC representa los **datos** de la aplicación y la **lógica de negocio** asociada. En la práctica, los datos suelen representarse mediante clases C# simples conocidas como **POCOs (Plain Old CLR Objects)**.

- ¿Qué es un POCO? Una clase C# "normal", sin heredar de clases base específicas del framework y enfocada en contener datos a través de propiedades públicas (generalmente autoimplementadas).
- Ejemplo (Modelo de Dominio):

```
// En la carpeta Models/  
public class Producto  
{  
    public int Id { get; set; } // Clave Primaria (quizás de la BD)  
    public string Nombre { get; set; }  
    public string Descripcion { get; set; }  
    public decimal Precio { get; set; } // Usar decimal para dinero  
    public int Stock { get; set; }  
    public bool Activo { get; set; }  
}
```

```
// Podría tener métodos de lógica de negocio aquí también
}
```

Esta clase *Producto* representa la entidad "Producto" tal como podría existir en nuestra base de datos o en la lógica central de la aplicación (nuestro *Modelo de Dominio*).

(1.3) ViewModels: Adaptando Datos para la Vista

A menudo, la información que una Vista necesita mostrar no coincide *exactamente* con la estructura de nuestro Modelo de Dominio.

- ¿Por qué?
 - La Vista podría necesitar mostrar solo un **subconjunto** de las propiedades (ej: un listado solo muestra Nombre y Precio).
 - La Vista podría necesitar mostrar datos **combinados** de varios Modelos de Dominio (ej: mostrar un Pedido junto con datos del Cliente y detalles de los Productos).
 - La Vista podría necesitar datos **formateados** de una manera específica (ej: una fecha como "dd/MM/yyyy", un precio con el símbolo "\$").
 - La Vista podría necesitar datos adicionales que no están en el Modelo de Dominio, como opciones para una lista desplegable.
 - Queremos **evitar pasar el Modelo de Dominio directamente a la Vista** para no acoplar fuertemente la Vista a la estructura interna de nuestros datos y para no exponer datos sensibles o innecesarios.
- La Solución: ViewModels (Modelos de Vista)
 - ¿Qué son? Son clases POCO diseñadas **específicamente para una Vista (o un conjunto pequeño de Vistas relacionadas)**. Contienen *exactamente* las propiedades que esa Vista necesita, en el formato que las necesita. Actúan como un intermediario o adaptador entre el Modelo de Dominio y la Vista.
 - Se suelen colocar en una carpeta separada llamada **ViewModels**.
- Ejemplo: Para una vista que crea un nuevo producto, el ViewModel podría ser muy similar al Modelo, pero quizás incluyendo validaciones o listas desplegables. Para una vista que *muestra* un producto, podría ser diferente:

```
// En la carpeta ViewModels/
public class ProductoDisplayViewModel
{
    public string NombreProducto { get; set; } // Podría tener un nombre
    diferente
    public string PrecioFormateado { get; set; } // El precio ya formateado
    como string
    public string DescripcionHtml { get; set; } // Quizás con algo de
    formato HTML
    public bool HayStock { get; set; } // Un booleano calculado
}
```

```
public class ProductoCreateViewModel
{
    // Propiedades necesarias para el formulario de creación
    public string Nombre { get; set; }
    public string Descripcion { get; set; }
    public decimal Precio { get; set; }
    public int StockInicial { get; set; }

    // Podría incluir aquí opciones para una categoría, por ejemplo
    // public List<SelectListItem> CategoriasDisponibles { get; set; }
}
```

- El Controlador es responsable de mapear los datos desde el Modelo de Dominio hacia el ViewModel antes de pasarlo a la Vista.

(1.4) Vistas Fuertemente Tipadas: Conectando el ViewModel

Ahora que tenemos un ViewModel, podemos crear Vistas que "sepan" qué tipo de datos esperar.

- **Paso 1: Pasar el ViewModel desde la Acción:** En lugar de retornar `View()` vacío o usar `ViewBag`, creamos una instancia del ViewModel, le cargamos los datos necesarios (mapeando desde el Modelo de Dominio si es necesario) y lo pasamos como argumento al método `View()`.

```
// En ProductoController.cs
public IActionResult Detalle(int id)
{
    // 1. Obtener el producto del Modelo de Dominio (lógica simulada)
    Producto producto = ObtenerProductoDesdeLaBaseDeDatos(id);
    if (producto == null) return NotFound(); // Si no existe

    // 2. Crear y mapear al ViewModel
    ProductoDisplayViewModel viewModel = new ProductoDisplayViewModel
    {
        NombreProducto = producto.Nombre,
        PrecioFormateado = producto.Precio.ToString("C"), // Formato de
moneda
        DescripcionHtml = ConvertirMarkdownAHtml(producto.Descripcion), //
Lógica de formato
        HayStock = producto.Stock > 0
    };

    // 3. Pasar el ViewModel a la Vista
    return View(viewModel); // MVC buscará Views/Producto/Detalle.cshtml
}
```

- **Paso 2: Declarar el Modelo en la Vista:** Al principio del archivo `.cshtml` (`Views/Producto/Detalle.cshtml` en este caso), usamos la directiva `@model` para especificar el tipo exacto del ViewModel que esta vista espera recibir. **Solo puede haber una directiva `@model` por archivo.**

```
@* Declara que esta vista espera un objeto ProductoDisplayViewModel *@
@model MiPrimeraAppMVC.ViewModels.ProductoDisplayViewModel

@{
    ViewData["Title"] = @Model.NombreProducto; // Usar propiedad del modelo
    para el título
}

<h1>Detalles de: @Model.NombreProducto</h1> <!-- Acceso con IntelliSense! --%>

<div>
    <strong>Precio:</strong> @Model.PrecioFormateado
</div>
<div>
    <strong>Descripción:</strong> @Html.Raw(Model.DescripcionHtml) <!--
    Mostrar HTML si es seguro --%>
</div>
<div>
    <strong>Disponible:</strong> @(Model.HayStock ? "Sí" : "No") <!--
    Expresión C# --%>
</div>
```

- **¡Las Ventajas!**

- **Seguridad de Tipos:** El compilador verifica que estés usando propiedades que realmente existen en el `ProductoDisplayViewModel`. Si escribes mal (`@Model.NomreProducto`), ¡tendrás un error de compilación, no en tiempo de ejecución!
- **IntelliSense:** El editor de Visual Studio te autocompleta los nombres de las propiedades después de `@Model.`, haciendo el desarrollo mucho más rápido y menos propenso a errores.
- **Refactorización:** Si cambias el nombre de una propiedad en la clase `ViewModel`, las herramientas de refactorización pueden ayudarte a actualizar todas sus referencias en las vistas.

(1.5) Model Binding: La Magia Automática

Ya vimos cómo MVC puede tomar valores de la URL y asignarlos a parámetros simples en las acciones. Este proceso se llama **Model Binding**. Funciona no solo para tipos simples, sino también para objetos complejos como nuestros ViewModels, especialmente cuando vienen de formularios.

- *¿Cómo funciona (conceptual)?* Cuando llega una petición HTTP (GET o POST), el *Model Binder* de ASP.NET Core examina los datos de la petición (valores de ruta, query string, cuerpo del formulario) y busca parámetros en la firma del método de acción o propiedades públicas en el tipo del parámetro complejo (nuestro ViewModel). Intenta **hacer coincidir los nombres** (ignorando mayúsculas/minúsculas) de los datos de la petición con los nombres de los parámetros/propiedades y **convierte los valores** al tipo esperado.
- *Ejemplos Vistos:*
 - URL `/Productos/Detalle/5` -> Acción `Detalle(int id)` -> El Model Binder toma el valor `5` del segmento de ruta `{id}` y lo asigna al parámetro `id`.
 - URL `/Productos/Buscar?termino=zapatillas&pagina=1` -> Acción `Buscar(string termino, int pagina)` -> El Model Binder toma `zapatillas` del query string `termino` y lo

asigna al parámetro `termino`, y toma `1` del query string `pagina` y lo asigna al parámetro `pagina`.

- Veremos cómo funciona con formularios POST en la siguiente sección. Es la misma idea: los `name` de los campos del formulario se intentan mapear a las propiedades del ViewModel.

Parte 2: Manejando la Entrada del Usuario - Formularios y Model Binding

(2.1) Formularios HTML: GET vs. POST

Recordemos el elemento `<form>`:

```
<form action="/Productos/Buscar" method="get"> ... </form>
<form action="/Productos/Crear" method="post"> ... </form>
```

- **method="get":**
 - Los datos de los campos del formulario (`<input name="nombre">`) se añaden a la URL especificada en `action` como pares `nombre=valor` en la *query string*.
 - **Ideal para:** Búsquedas, filtros, paginación. Operaciones que **no modifican** datos en el servidor (idempotentes).
 - **Limitaciones:** Longitud máxima de URL, datos visibles. No usar para información sensible.
- **method="post":**
 - Los datos del formulario se envían en el **cuerpo** de la petición HTTP, no en la URL.
 - **Ideal para:** Crear nuevos datos, actualizar existentes, eliminar datos, enviar contraseñas o cualquier información sensible o de gran tamaño.

(2.2) Manejando Formularios en Acciones MVC

El patrón común para manejar un formulario que modifica datos (ej: crear o editar) es tener **dos métodos de acción** en el Controlador con el **mismo nombre**, pero uno que maneje la petición *GET* y otro la petición *POST*:

1. Acción GET ([HttpGet] opcional pero recomendado para claridad):

- *Propósito:* **Mostrar el formulario vacío** (o con datos existentes si es para editar).
- *Firma Típica:* `public IActionResult Crear()` o `public IActionResult Editar(int id)`
- *Lógica:* Usualmente, crea una instancia del ViewModel (vacía para Crear, o cargada con datos de la BD para Editar) y la pasa a la vista: `return View(viewModel);`.

2. Acción POST ([HttpPost] obligatorio):

- *Propósito:* **Recibir y procesar los datos enviados** por el formulario cuando el usuario hace clic en el botón de submit.
- *Firma Típica:* `public IActionResult Crear(MiViewModel viewModel)` o `public IActionResult Editar(int id, MiViewModel viewModel)`
- *Model Binding en Acción:* ASP.NET Core automáticamente intentará **crear una instancia** de `MiViewModel` y **rellenar sus propiedades** con los valores que vienen del formulario (buscando por los atributos `name` de los inputs/selects/textareas que coincidan con los nombres de las propiedades del ViewModel).
- *Lógica:*
 - **Validar** los datos recibidos en `viewModel` (ver Parte 4).
 - Si son válidos, interactuar con el Modelo (ej: guardar en la BD).

- Decidir qué respuesta dar (usualmente una redirección - Patrón PRG).
- Si **no** son válidos, volver a mostrar el **mismo formulario**, pasando de vuelta el objeto `viewModel` (que contiene los datos que el usuario ya ingresó y los errores de validación) para que el usuario corrija.

- *Ejemplo de Acciones Crear:*

```
// En ProductoController.cs

// GET: /Producto/Crear
[HttpGet] // Opcional si solo hay una acción 'Crear' sin HttpPost
public IActionResult Crear()
{
    // Simplemente muestra el formulario vacío
    ProductoCreateViewModel viewModel = new ProductoCreateViewModel();
    // Podríamos inicializar alguna propiedad si fuera necesario
    return View(viewModel); // Busca Views/Producto/Crear.cshtml
}

// POST: /Producto/Crear
[HttpPost]
// [ValidateAntiForgeryToken] // Importante añadir esto por seguridad (ver
// Tag Helpers)
public IActionResult Crear(ProductoCreateViewModel viewModel)
{
    // 'viewModel' llega aquí poblado gracias al Model Binding
    // con los datos enviados desde el formulario <form method="post">

    // PASO 1: Validar el modelo (lo veremos en detalle después)
    if (ModelState.IsValid) // Propiedad que verifica si hubo errores de
binding o validación
    {
        // Modelo válido, proceder a guardar (lógica simulada)
        Producto nuevoProducto = new Producto {
            Nombre = viewModel.Nombre,
            Descripcion = viewModel.Descripcion,
            Precio = viewModel.Precio,
            Stock = viewModel.StockInicial,
            Activo = true
        };
        GuardarProductoEnLaBaseDeDatos(nuevoProducto);

        // Patrón PRG: Redirigir a otra acción (ej: Index) tras éxito
        return RedirectToAction("Index");
    }
    else
    {
        // Modelo NO válido. Hay errores.
        // Volver a mostrar el formulario 'Crear.cshtml', pasando de vuelta
        // el 'viewModel' para que los campos muestren los datos que
        // el usuario ya ingresó y se puedan mostrar los mensajes de error.
        return View(viewModel);
    }
}
```

```
}
}
```

(2.3) El Patrón Post-Redirect-Get (PRG)

- *Problema:* Si la acción POST, después de guardar los datos, simplemente retorna `View()`, la URL en el navegador sigue siendo la de la acción POST. Si el usuario presiona F5 (refrescar), el navegador preguntará si quiere "reenviar los datos del formulario". Si el usuario acepta, ¡la acción POST se ejecuta de nuevo!, potencialmente creando registros duplicados o realizando acciones no deseadas.
- *Solución (PRG):*
 1. El usuario envía el formulario (**POST**).
 2. El servidor procesa los datos. Si son válidos y la operación es exitosa:
 3. El servidor **NO** devuelve una vista directamente, sino que envía una respuesta de **Redirección** (HTTP 302 o 303) a una URL diferente (usualmente la página que muestra la lista o el detalle del ítem recién creado/editado). Se hace con `RedirectToAction("NombreAccion", [Opcional: "NombreControlador"], [Opcional: new { id = valorId }])`.
 4. El navegador recibe la redirección y automáticamente realiza una **nueva petición GET** a la URL indicada.
 5. El servidor maneja esta petición GET y devuelve la vista correspondiente.
 - *Ahora*, si el usuario presiona F5, simplemente repetirá la última petición GET, lo cual es seguro (no modifica datos).
 - Por eso en el ejemplo anterior, si `ModelState.IsValid`, hacemos `return RedirectToAction("Index");`.

Parte 3: Simplificando la Creación de Formularios - Tag Helpers

Escribir formularios HTML manualmente (`<input name="Nombre" id="Nombre" value="...">`) es tedioso y propenso a errores, especialmente al tener que sincronizar los atributos `name` e `id` con las propiedades del ViewModel y al manejar el valor (`value`) correctamente. ASP.NET Core introduce los **Tag Helpers** para simplificar enormemente esto.

(3.1) ¿Qué son los Tag Helpers?

- Son componentes del lado del servidor, procesados por Razor en los archivos `.cshtml`.
- Se escriben de forma muy similar a etiquetas HTML estándar, pero usan atributos especiales prefijados con `asp-` (o a veces actúan sobre etiquetas HTML existentes).
- Razor los procesa y los convierte en HTML real antes de enviar la página al navegador.
- *Ventajas sobre escribir HTML manual o usar los antiguos @Html Helpers:*
 - **Sintaxis Limpia:** Muy parecida a HTML, más fácil de leer y escribir para diseñadores y desarrolladores.
 - **Integración Fuerte con el Modelo:** Usan la información del ViewModel (a través del atributo `asp-for`) para generar automáticamente los atributos `id`, `name`, `value` correctos, e incluso el `type` del input basado en el tipo de dato de la propiedad.
 - **Menos Errores:** Reducen drásticamente los errores por tipeo en nombres de atributos.
 - **Validación Integrada:** Generan automáticamente los atributos `data-val-*` necesarios para la validación del lado del cliente basada en las DataAnnotations del modelo.
- **Habilitación:** Para que funcionen, la directiva `@addTagHelper *`, `Microsoft.AspNetCore.Mvc.TagHelpers` debe estar presente en el archivo

Views/_ViewImports.cshtml. Los proyectos MVC creados desde la plantilla de Visual Studio ya la incluyen.

(3.2) Tag Helpers Esenciales para Formularios

Estos reemplazan o mejoran las etiquetas HTML estándar para formularios:

- **<form asp-controller="NombreCtrl" asp-action="NombreAccion" [asp-route-{parametro}="valor"] [method="post"]>**
 - *Genera:* `<form action="/Url/Correcta/Generada/Por/Router" method="post"> ... </form>`
 - **asp-controller, asp-action:** Le indican a qué acción de qué controlador debe apuntar el **action** del formulario. Razor usa el sistema de routing para generar la URL correcta.
 - **asp-route-{parametro}:** Permite pasar parámetros de ruta adicionales si la acción los requiere (ej: `asp-route-id="@Model.Id"` para un formulario de edición).
 - **method:** Especifica el método HTTP (si se omite, por defecto es `post`).
 - **Seguridad:** Automáticamente añade un campo oculto (`<input type="hidden" name="__RequestVerificationToken" ...>`) con un token anti-falsificación. Para que funcione, la acción POST en el controlador debe tener el atributo `[ValidateAntiForgeryToken]`. Esto ayuda a prevenir ataques CSRF.
- **<label asp-for="NombrePropiedad">**
 - *Genera:* `<label for="NombrePropiedad">Texto de la Etiqueta</label>`
 - **asp-for:** Especifica la propiedad del `@model` a la que está asociada esta etiqueta.
 - Genera el atributo **for** correctamente, vinculándolo al **id** del input generado por `<input asp-for="NombrePropiedad">`.
 - El texto de la etiqueta se toma del atributo `[Display(Name="...")]` de la propiedad, o del nombre de la propiedad si no hay `DisplayAttribute`.
- **<input asp-for="NombrePropiedad">**
 - *Genera:* `<input id="NombrePropiedad" name="NombrePropiedad" type="..." value="..." data-val="true" ...>`
 - **asp-for:** La vincula a la propiedad del modelo.
 - Genera **id** y **name** automáticamente. ¡Adiós errores de tipeo!
 - Genera el **type** apropiado según el tipo de dato de la propiedad (`text`, `number`, `password`, `email`, `date`, `datetime-local`, `checkbox`, etc.). Para `bool`, genera `type="checkbox"`.
 - Genera el **value** basado en el valor actual de `@Model.NombrePropiedad`. Esto es crucial para que los formularios de edición muestren los datos existentes y para que los formularios reenviados por validación fallida conserven los datos ingresados por el usuario.
 - Genera atributos `data-val-*` si la propiedad tiene DataAnnotations de validación.
- **<textarea asp-for="NombrePropiedad">**
 - *Genera:* `<textarea id="NombrePropiedad" name="NombrePropiedad" ...>@Model.NombrePropiedad</textarea>`
 - Similar a `input`, pero para texto multilínea. El valor se coloca *dentro* de las etiquetas.
- **<select asp-for="NombrePropiedad" asp-items="FuenteDeOpciones">**
 - *Genera:* `<select id="NombrePropiedad" name="NombrePropiedad"> <option ...> </select>`
 - **asp-for:** Vincula la propiedad del modelo que almacenará el *valor* (**value**) de la opción seleccionada.

- **asp-items:** **Requiere** un `IEnumerable<SelectListItem>`. Cada `SelectListItem` tiene propiedades `Value` (lo que se envía al servidor) y `Text` (lo que ve el usuario). Esta colección se **prepara en el Controlador** (normalmente en la acción GET que muestra el formulario) y se pasa a la Vista (ya sea como una propiedad del ViewModel o usando `ViewBag`). El Tag Helper genera las etiquetas `<option>` a partir de esta colección, seleccionando automáticamente la opción cuyo `Value` coincida con el valor actual de `@Model.NombrePropiedad`.

```
// En el ViewModel:
// public string CategoriaSeleccionada { get; set; } // Para asp-for
// public List<SelectListItem> CategoriasOptions { get; set; } // Para asp-
// items

// En la Acción GET del Controlador:
// viewModel.CategoriasOptions = ObtenerCategoriasComoSelectListItems();
// return View(viewModel);

// En la Vista:
// <select asp-for="CategoriaSeleccionada" asp-
// items="Model.CategoriasOptions">
//     <option value="">-- Seleccione Categoría --</option> <!-- Opción por
// defecto opcional --%>
// </select>
```

(3.3) Práctica: Convirtiendo a Tag Helpers

El ejercicio consiste en tomar un formulario HTML creado manualmente (de la clase anterior) y reescribirlo utilizando los Tag Helpers (`form`, `label`, `input`, etc.) vinculados a las propiedades de un ViewModel. Es fundamental verificar el HTML generado usando las Herramientas de Desarrollador del navegador para entender qué está haciendo cada Tag Helper.

Parte 4: Asegurando la Calidad de los Datos y Vistas Avanzadas

(4.1) Validación de Datos: ¡No Confíes en el Usuario!

Es **fundamental** validar cualquier dato que provenga del usuario o de fuentes externas antes de procesarlo o guardarlo.

- *¿Por qué validar?*
 - **Experiencia de Usuario (UX):** Guiar al usuario para que ingrese datos correctos.
 - **Integridad de Datos:** Asegurar que los datos almacenados sean válidos, consistentes y cumplan las reglas del negocio.
 - **Seguridad:** Prevenir ataques o errores causados por datos maliciosos o inesperados.
- **Validación Lado Cliente vs. Lado Servidor:**
 - **Cliente (Navegador, con JavaScript):** Rápida, mejora la UX dando feedback inmediato. **PERO** puede ser desactivada o evitada por el usuario. **NUNCA** debe ser la única validación.
 - **Servidor (En el Controlador C#): Obligatoria e indispensable.** Es la única validación segura y confiable, ya que se ejecuta en nuestro código antes de procesar los datos.

(4.2) DataAnnotations: Definiendo Reglas en el Modelo/ViewModel

ASP.NET Core facilita la definición de reglas de validación directamente en las propiedades de nuestras clases Modelo o (preferiblemente) ViewModel usando **Atributos** del *namespace*

`System.ComponentModel.DataAnnotations`.

```
using System.ComponentModel.DataAnnotations; // Namespace necesario

public class ProductoCreateViewModel
{
    [Required(ErrorMessage = "El nombre del producto es obligatorio.")] // Campo
requerido
    [StringLength(100, MinimumLength = 3, ErrorMessage = "El nombre debe tener
entre 3 y 100 caracteres.")] // Longitud
    [Display(Name = "Nombre del Producto")] // Texto para el <label>
    public string Nombre { get; set; }

    [StringLength(500, ErrorMessage = "La descripción no puede exceder los 500
caracteres.")]
    public string Descripcion { get; set; } // No es 'Required', puede ser
nulo/vacío

    [Required(ErrorMessage = "Debe ingresar un precio.")]
    [Range(0.01, 1000000.00, ErrorMessage = "El precio debe ser mayor a 0 y menor
a 1,000,000.")] // Rango numérico
    [DataType(DataType.Currency)] // Ayuda a Tag Helpers a elegir formato (y a
veces input type)
    public decimal Precio { get; set; }

    [Required(ErrorMessage = "El stock es requerido.")]
    [Range(0, int.MaxValue, ErrorMessage = "El stock no puede ser negativo.")]
    [Display(Name = "Stock Inicial")]
    public int StockInicial { get; set; }

    // Ejemplo para email (en otra clase quizás):
    // [Required]
    // [EmailAddress(ErrorMessage = "Ingresa un formato de email válido.")]
    // public string Email { get; set; }

    // Ejemplo para confirmar contraseña (en otra clase):
    // [Required]
    // [DataType(DataType.Password)]
    // public string Password { get; set; }

    // [DataType(DataType.Password)]
    // [Compare("Password", ErrorMessage = "Las contraseñas no coinciden.")]
    // [Display(Name = "Confirmar Contraseña")]
    // public string ConfirmPassword { get; set; }
}
```

- Estos atributos son leídos tanto por el sistema de Model Binding en el servidor como por los Tag Helpers y el JavaScript de validación en el cliente.

(4.3) Validación en el Servidor: `ModelState.IsValid`

Como vimos antes, en la acción POST, después de que el Model Binder intenta poblar el ViewModel, debemos verificar si el proceso fue exitoso y si se cumplieron todas las reglas de DataAnnotations.

```
[HttpPost]
[ValidateAntiForgeryToken] // No olvidar por seguridad
public IActionResult Crear(ProductoCreateViewModel viewModel)
{
    if (ModelState.IsValid) // ¡La comprobación clave!
    {
        // Si IsValid es true, significa que el Model Binding tuvo éxito Y
        // todas las reglas de DataAnnotations se cumplieron.

        // ... Mapear viewModel a Modelo de Dominio y guardar en BD ...

        return RedirectToAction("Index"); // PRG
    }

    // Si IsValid es false, hubo errores. ModelState contiene los detalles.
    // Volver a mostrar la misma vista, pasando el mismo viewModel.
    // La vista usará los datos en viewModel para repoblar los campos
    // y usará los errores en ModelState para mostrar los mensajes.

    // ¡Recordar recargar datos para SelectList si los usaste en
    ViewBag/ViewModel!
    // ViewBag.CategoriasOptions = ...;

    return View(viewModel);
}
```

- `ModelState` es un diccionario que contiene los errores. Puedes añadir errores manualmente también (`ModelState.AddModelError("NombrePropiedad", "Mensaje de error personalizado");`).

(4.4) Mostrando Errores en la Vista: Tag Helpers de Validación

Cuando la acción POST devuelve la vista porque `ModelState.IsValid` fue falso, necesitamos mostrarle al usuario qué estuvo mal. Los Tag Helpers de validación se conectan automáticamente con `ModelState`.

- `<span asp-validation-for="NombrePropiedad" class="text-danger"></span>`
 - *Genera:* Un `<span>`. Si existe un error asociado a `NombrePropiedad` en `ModelState`, inserta el mensaje de error dentro del `span`.
 - *Ubicación:* Colócalo justo después del `<input asp-for>` o `<select asp-for>` correspondiente.
 - `class="text-danger"`: Clase CSS (común en Bootstrap) para darle estilo (ej: color rojo).
- `<div asp-validation-summary="All | ModelOnly" class="text-danger"></div>`
 - *Genera:* Un `<div>` que muestra una lista `<ul>` con los mensajes de error.
 - `ModelOnly`: Muestra solo los errores que no están asociados a una propiedad específica (los que añadiste manualmente a `ModelState` sin clave o con clave vacía).

- **ALL**: Muestra **todos** los errores, incluyendo los de nivel de propiedad. Útil para tener un resumen al principio del formulario.
- *Ubicación*: Usualmente al principio del `<form>`.
- *Ejemplo en la Vista (Crear.cshtml)*:

```
@model MiPrimeraAppMVC.ViewModels.ProductoCreateViewModel

<form asp-action="Crear" method="post">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div> <!--
Resumen de errores no ligados a campos --%>

    <div class="form-group">
        <label asp-for="Nombre"></label>
        <input asp-for="Nombre" class="form-control">
        <span asp-validation-for="Nombre" class="text-danger"></span> <!--
Error para Nombre --%>
    </div>
    <div class="form-group">
        <label asp-for="Precio"></label>
        <input asp-for="Precio" class="form-control">
        <span asp-validation-for="Precio" class="text-danger"></span> <!--
Error para Precio --%>
    </div>
    <!-- ... otros campos con sus validation-for ... --%>

    <button type="submit" class="btn btn-primary">Crear Producto</button>
</form>

<!-- Importante incluir los scripts de validación cliente --%>
@section Scripts {
    @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
}
```

(4.5) Validación en el Lado del Cliente (Automática)

Una de las grandes ventajas de usar DataAnnotations y Tag Helpers es que la validación básica del lado del cliente (en el navegador) se habilita casi sin esfuerzo adicional.

- *¿Cómo?*
 1. Los **DataAnnotations** en el ViewModel definen las reglas.
 2. Los **Tag Helpers** (`input asp-for`, etc.) leen esas DataAnnotations y generan atributos HTML `data-val-*` en los inputs (ej: `data-val="true"`, `data-val-required="El nombre es obligatorio."`).
 3. Se incluyen las **librerías JavaScript** adecuadas en la página (jQuery, jQuery Validate, jQuery Validate Unobtrusive - la plantilla MVC usualmente configura esto a través de `_ValidationScriptsPartial.cshtml`).
 4. El script de **jQuery Validate Unobtrusive** detecta los atributos `data-val-*` y configura **jQuery Validate** para aplicar esas reglas en el navegador *antes* de que el formulario se envíe. Muestra y oculta los mensajes en los `<span>` generados por `asp-validation-for`.

- *Resultado:* El usuario obtiene feedback inmediato si un campo es requerido, si un email no tiene formato válido, etc., sin necesidad de enviar el formulario al servidor. Mejora mucho la experiencia.
- **¡Recordatorio!** La validación cliente es una conveniencia, **la validación en el servidor** (`ModelState.IsValid`) **es la única fuente de verdad y es obligatoria por seguridad.**

(4.6) Vistas Avanzadas: Reutilización y Organización

A medida que las vistas crecen, necesitamos formas de organizarlas y reutilizar partes comunes.

- **Secciones Razor (@section)**

- *Propósito:* Permitir que una Vista de contenido (ej: `Index.cshtml`) "inyecte" bloques de código HTML o Razor en lugares específicos definidos en su archivo `_Layout.cshtml`.
- *Definición en Layout (`_Layout.cshtml`):* Se usa `@RenderSection("NombreDeLaSeccion", required: false)`. El `required: false` significa que las vistas no están obligadas a definir esta sección.

```
// En _Layout.cshtml, usualmente al final del <body>
<script src="~/lib/jquery/jquery.min.js"></script>
@await RenderSectionAsync("Scripts", required: false) <!-- Punto de
inyección llamado "Scripts" -->
</body>
```

- *Definición en la Vista de Contenido (`MiVista.cshtml`):* Se usa la directiva `@section`.

```
@model MiViewModel

<h1>Contenido Principal</h1>
<!-- ... resto del HTML ... -->

@section Scripts {
    <!-- Este bloque se renderizará donde estaba
    @RenderSection("Scripts") -->
    <script src="~/js/mi-script-especifico-para-esta-pagina.js">
</script>
    <script>
        console.log("JavaScript solo para MiVista cargado!");
    </script>
}
```

- *Uso Común:* Incluir archivos JavaScript o bloques `<style>` específicos de una página sin tener que cargarlos en todas las páginas a través del Layout.
- **Vistas Parciales (Partial Views)**
 - *Propósito:* Encapsular un fragmento de interfaz de usuario (HTML y Razor) que se necesita reutilizar en múltiples vistas o para simplificar una vista muy compleja. Son como "controles" o "componentes" reutilizables.
 - *Creación:* Son archivos `.cshtml` normales, usualmente ubicados en `Views/Shared` y con un nombre que empieza con `_` (convención), ej: `_TarjetaProducto.cshtml`. Pueden tener o no

una directiva `@model` si necesitan recibir datos específicos.

- *Renderizado dentro de otra Vista:* Se usa el Tag Helper `<partial>`:

```
<!-- Renderizar una vista parcial simple --%>
<partial name="_MenuLateral" />

<!-- Renderizar pasando un modelo específico a la parcial --%>
<!-- (Asume que _TarjetaProducto.cshtml tiene @model
ProductoResumenViewModel) --%>
@foreach (var producto in Model.ListaProductos)
{
    <partial name="_TarjetaProducto" model="producto" />
}

<!-- Forma alternativa (más antigua) con Html Helper --%>
@await Html.PartialAsync("_OtraParcial", Model.AlgunaPropiedad)
```

- *¿Para qué?* Evitar duplicar código HTML para elementos comunes (ej: mostrar una dirección, una tarjeta de producto, un widget de login, un elemento de lista complejo).

Conclusión

Este eje temático nos ha llevado al corazón de la interacción en MVC. Hemos aprendido a representar nuestros datos con Modelos y a adaptarlos para la interfaz con ViewModels, permitiendo Vistas fuertemente tipadas. Dominamos el flujo de formularios usando GET y POST, y vimos cómo el Model Binding nos facilita enormemente la recepción de datos. Los Tag Helpers nos dieron una sintaxis limpia y potente para generar HTML de formularios integrado con nuestro modelo. Finalmente, aprendimos a implementar validaciones robustas con DataAnnotations (tanto en servidor como en cliente) y a organizar nuestras Vistas con Secciones y Vistas Parciales. Con estas herramientas, podemos construir interfaces web interactivas, seguras y mantenibles con ASP.NET Core MVC.

Síntesis de una oración: Este material detalla cómo usar Modelos y ViewModels en ASP.NET Core MVC para crear Vistas fuertemente tipadas, manejar formularios con GET/POST y Model Binding, simplificar la generación de UI con Tag Helpers, implementar validaciones robustas con DataAnnotations y organizar las vistas con secciones y parciales.

Síntesis de 5 puntos importantes:

1. Usar **ViewModels** específicos para cada Vista es preferible a *ViewBag/ViewData*, permitiendo **Vistas fuertemente tipadas** (`@model`) con IntelliSense y chequeo en compilación.
2. El **Model Binding** mapea automáticamente datos de la petición HTTP (ruta, query string, formulario) a parámetros de Acción y propiedades de ViewModel, basándose en la coincidencia de nombres.
3. Los **Tag Helpers** (`asp-for`, `asp-action`, etc.) simplifican la creación de formularios HTML, integrándolos con el modelo y habilitando funcionalidades como la validación y anti-falsificación.
4. La **Validación** se define con **DataAnnotations** en el ViewModel/Modelo y se verifica **obligatoriamente** en el servidor (`ModelState.IsValid`), complementándose con validación cliente (JavaScript) para mejor UX y mostrando errores con Tag Helpers (`asp-validation-for`).

5. Las **Secciones Razor** (`@section`) y **Vistas Parciales** (`<partial>`) ayudan a organizar y reutilizar código dentro de las Vistas y Layouts.

51888